

```
nonce = Crypto_HMAC(key = key, message = counter)[0..(CRYPTO_AEAD_NONCE_LENGTH_BYTES-1)]
```

4.22.3.2. Authenticated encryption

The payload of the Check-In message SHALL be encrypted using the [AEAD](#) function. The inputs of the algorithm SHALL be the generated [nonce](#), the key and the plaintext to encrypt. The additional data field SHALL NOT be used.

```
Ciphertext = Crypto_AEAD_GenerateEncrypt(  
    K = key  
    P = plaintext,  
    A = "",  
    N = nonce)
```

4.22.3.3. Client Counter Validation

When receiving a Check-In message, the client SHALL verify the received counter by executing the following steps.

1. Subtract the stored starting value of the counter from the value provided in the Check-In message. The subtraction SHALL be done as unsigned integers mod 2^{32} . This ensures that even if the subtraction rolls over, it will still produce the correct offset.
2. If the stored last known offset from the stored starting value of the counter is smaller than the value from step 1, the verification SHALL return TRUE.
3. Otherwise the verification SHALL return FALSE.

4.22.3.4. Key Refreshing

A given key is only valid for one set of values of the [Check-In Counter](#). A key becomes invalid once all the [Check-In Counter](#) values for that key have been used. The tracking of the used Check-In counter values is done as part of the [decryption procedure](#).

To avoid reuse of a nonce value with a given key, the key (shared secret) needs to be refreshed before all the valid [Check-In Counter](#) values for that key have been used. If a [Check-In Counter](#) value is reused with the same key, the Check-In messages reusing the counter will be discarded. To avoid the situation where the relationship is broken because of a rollover, we are prescribing when a client should refresh its entry.

Key Refresh Validation

When a client receives a Check-In message with a [Check-In Counter](#) value indicating that 2^{31} counter values have been used, the client SHALL refresh its entry with a new key. The client SHALL determine if a key refresh is necessary by executing the following steps.

1. Check whether the stored counter offset is greater than or equal to 2^{31} .